CS 435: Cloud Networking **Spring 2026**

Machine Problem 4: Load Balancing Contest

Lead TAs: Gerard Matthew *Due: 11:59 pm Thursday, May 14, 2026*

1 Introduction

In this machine problem, you will implement an application-layer load balancer. A load balancer implemented at the application layer is a proxy that accepts incoming client requests and forwards them to backend servers (as illustrated in Figure 1) while optimizing for high throughput and low latency by **load balancing** (i.e., effectively spreading) requests across the backend servers. The load balancer should provide good performance even under a variety of conditions – dynamic server capacity fluctuations, intermittent unavailability, and API availability constraints (i.e., a given backend server may support only a subset of the API endpoints).

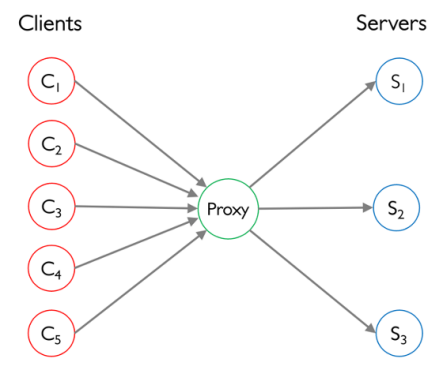

Figure 1: Client requests are sent to the proxy, which load balances them to the backend servers. The proxy then returns the server responses to the clients.

Effective load balancing is critical for real-world large-scale distributed systems, for maintaining performance, reliability, and resource utilization. It is challenging as it involves monitoring the state of the system in real time (including potentially multiple factors like server utilization, capacity, processing speed, availability, and queue build-up), and then dynamically assigning requests to appropriate backend servers in a way that minimizes overall response time. Therefore, as a core part of this MP, you will need to devise a proxy with an efficient load balancing strategy which ensures requests are serviced quickly without overloading any single server. In addition, your proxy must incorporate **fault tolerance mechanisms** (e.g., what is sometimes called "circuit breaking") to handle server downtimes or slow responses.

To encourage friendly competition, we've also introduced a ☆ leaderboard ☆ for this MP at `https://netarch.github.io/cs-435-contestboard/` which displays the top performance scores achieved. You may choose whether to participate in the leaderboard, as discussed in Section 4.2.

2 Load Balancer System Overview

As shown in Figure 1, the system consists of clients, a proxy, and servers. You are provided a client and server implementation and your task is to design the proxy. Next, we describe the responsibilities of each of these components.

2.1 Client

The client is provided in the MP package, and you should not modify it. It operates as follows:

- Each client simulates traffic from a batch of users by generating HTTP requests. There may be one or more clients running concurrently.
- Clients send requests to the proxy using HTTP/2 (via the `httpx` library) by invoking specific endpoints (e.g., `/api1`, `/api2`). An example URL is given below:

```
http://127.0.0.1:8000/proxy/api1/
```

- Finally, clients collect the responses back from the proxy and calculate statistics such as throughput and latency of requests.

2.2 Proxy

Starter code for the proxy is provided in the MP package, and your task is to improve it. The proxy should operate as follows:

- At startup, load the configuration, which we call the **capability file**. This file is given via a –capability_file argument, and lists which servers exist and which APIs they support. Each line in the file has the following format:

```
<server_address> <comma-separated list of APIs>
```

  Example capability file:

```
127.0.0.1:8001 api2
127.0.0.1:8002 api1
127.0.0.1:8003 api1,api2
```

  Your proxy must parse this file and maintain an internal mapping of which servers support which APIs. When forwarding a request from a client, the proxy must choose a server that supports the specific API the client is trying to access.
- Listen on a designated IP address and port: `127.0.0.1:8000`.
- For each incoming client request:
  - Select a backend server that supports the requested API and is currently available to receive requests.
  - Forward the request, without modifying its contents, to the selected server.
  - Relay the server's response back to the client.
- Probe backend servers periodically to track their availability and supported APIs.

- Temporarily stop sending requests to a failed server and resume sending upon detection of server recovery.

Note that the brains of the proxy lie in how to select the specific server for each request (the load balancing strategy) and how to track server availability or slowdowns (the fault tolerance strategy).

The starter code has a proxy implementation that correctly forwards requests to backend servers and relays their responses back to clients. However, it follows a simplistic policy of routing all traffic to the first server, so performance will be very poor. Additionally, it lacks mechanisms for handling server failures, such as shutdowns or slowdowns.

Your responsibility is to enhance this code, implementing better load balancing and failure handling mechanisms, to transform it into a fully functioning, high-performance proxy. You are free to design your own mechanisms, and the autograder will benchmark the resulting performance to see how well your proxy **maximizes throughput** and **minimizes latency** of requests. Clever mechanisms may earn higher performance scores.

2.3 Server

The server is provided in the MP package, and you should not modify it. It operates as follows:

- Each backend server listens on its designated `IP:port`, exposing multiple APIs. Each server may implement a subset of these APIs (e.g., server 1 may only expose an API1 endpoint whereas server 2 may expose both APIs).
- Servers could experience intermittent failures, such as shutdowns or slowdowns which could lead to requests failing or getting delayed.
- If the server is available, it will process the client's request. The processing delay may be nondeterministic, and will increase if the server is overloaded. The server sends a response in the following format:

```
"server_response": {
    "api": api1,
    "port": 8003,
    "message": "Response from api1 on port 8003"
}
```

- Each server also expects a parameter called `'client_key'` in the request sent from the client (the proxy should not modify it in any way).

3 Execution and Testing

Starter code is provided in MP4.zip with the following contents:

```
.
├── capability-files
│   └── capability_file.txt
├── client.py
├── proxy.py
├── server.py
├── requirements.txt
└── testcases
    ├── test_case_0a.txt
    └── test_case_0b.txt
```

To design, improve, and test your proxy, the entire system can be run locally on the course VM by installing the required dependencies and launching the system components. For deterministic results, we recommend testing within the course VM which provides resource isolation. To do so, open three terminal windows within the VM (as shown in Figure 4) and launch the system components in the following order: proxy, server, and client. It's crucial to follow this order since the server and proxy must be running before the client issues requests. For test scenarios where the server has simulated faults (e.g., slowdown or shutdown) at specific times, please start the server and client in quick succession.

3.1 Installing dependencies

Since `Python3` and `Pip` are already installed in the course VM, you can run the following command to install dependencies using the requirements.txt file.

```
pip install -r requirements.txt
```

3.2 Launching Backend Servers

```
python3 server.py --test_case_file test-cases/test_case_0a.txt
```

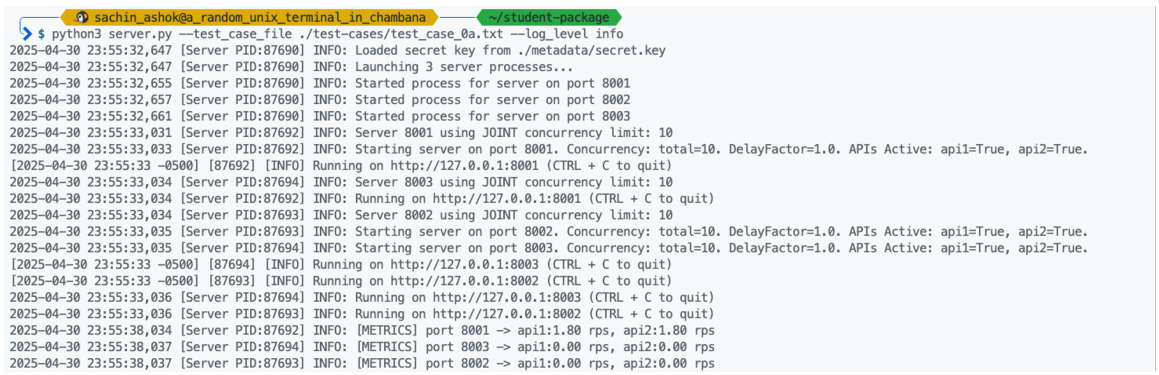

Figure 2: The terminal window running the server process.

3.3 Launching Your Proxy

```
python3 proxy.py --capability_file capability-files/capability_file.txt --port 8000
```

Figure 3: The terminal window running the proxy process.

3.4 Launching the Client Workload

```
python3 client.py --test_case_file test-cases/test_case_0a.txt \
--proxy_url http://127.0.0.1:8000/proxy/ --log_level info --plot
```

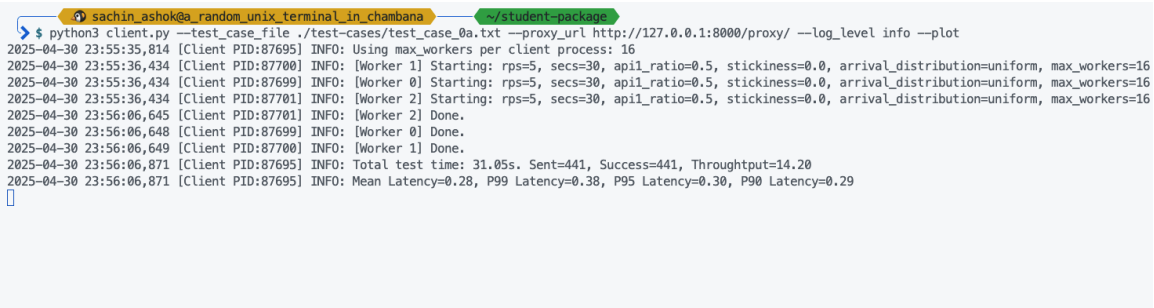

Figure 4: The terminal window running the client process.

3.5 Test Cases

Two sample test cases are provided to evaluate if your starter code works. You should find that the provided proxy provides "good performance" to clients in the case of test case #1 (`test_case_0a.txt`) while providing "bad performance" for clients in test case #2 (`test_case_0b.txt`) due to increased load sent by the clients in the latter case.

We encourage you to create new test cases to try out different scenarios. (The autograder will have more interesting scenarios than just the tests in the starter code!) You can also modify the provided `client.py` or `server.py` files, which may be sometimes useful for debugging, but keep in mind that the autograder will use the unmodified versions.

To understand the tests and help you know how to write your own tests, here is a description of the test case format. Each test case specifies the following details (example files are shown):

- Server configuration:

```
# Server: port delay_factor min_api1 max_api1 min_api2 max_api2
# thread_count1 thread_count2 api1_active api2_active
# [incident_type start_t duration_t [extra_delay]]
Server: 8001 1.0 0.2 0.3 0.5 0.6 5 5 1 1
Server: 8002 1.0 0.2 0.3 0.5 0.6 5 5 1 1 shutdown 10 5
Server: 8003 1.0 0.2 0.3 0.5 0.6 5 5 1 1 slowdown 15 10 0.5
```

- Client configuration:

```
# Client: rps duration api1_ratio stickiness distribution
Client: 30 30 0.5 0 deterministic
Client: 40 30 0.5 0 exponential
Client: 50 30 0.5 0 deterministic
```

Here is what the above example files mean:

- **Server Configuration:** There are three servers. They operate at the same capacity, since the delay factor that determines their performance is set to 1.0 for all servers in this case (a higher delay factor results in slower servers). Requests to API1 experience a processing delay between 200 ms and 300 ms, while requests to API2 incur delays between 500 ms and 600 ms. Each server has a total of C = thread_count1 + thread_count2 threads ($5 + 5 = 10$ total threads in this case) to process requests in parallel. If concurrent requests exceed this capacity, they are queued at the server, resulting in non-zero queuing delays. Additionally, Server 2 goes offline at t=10s for 5 seconds, while Server 3 experiences a slowdown starting at t=15s, lasting 10 seconds. During this slowdown, all requests processed incur an additional 0.5 seconds latency (due to the final `0.5` on the last line of the server configuration).
- **Client Configuration:** The test case involves three clients (or users) generating traffic at rates of 30, 40, and 50 requests per second (RPS), respectively. Each client invokes API1 with probability `api1_ratio` and API2 with probability $1 -$ `api1_ratio` (set to 0.5 for both in this case). Clients 1 and 3 send traffic at a deterministic rate, with request arrivals governed by a deterministic inter-arrival distribution. In contrast, Client 2 exhibits bursty behavior, with request arrivals following an exponential inter-arrival distribution, while maintaining the same overall RPS as Clients 1 and 3.

3.6 Test Wrap-Up Phase

After all clients in the test case have completed execution, the `client.py` script calculates the mean and 95th percentile latency, as well as the mean throughput achieved by the clients. It then computes the overall score based on the scoring scheme outlined in Section 4. To see detailed logs from the client, proxy, and the server, the `--log_level` argument can be adjusted (e.g., debug, info, warning, critical) when invoking them. The `client.py` script can also be passed the argument `--plot` (as described in Section 3.4) to generate both a table and a plot (shown in Figure 5 below) visualizing the results of the test case.

Latency/ Throughput Performance Summary

Metric	Value
Total Requests Sent	441
Total Requests Succeeded	441
Total Requests Dropped	0
Throughput	14.13 req/s
Minimum Latency	0.2222s
Mean Latency	0.2490s
90th Percentile Latency	0.2692s
95th Percentile Latency	0.2826s
99th Percentile Latency	0.3045s
Maximum Latency	0.3236s

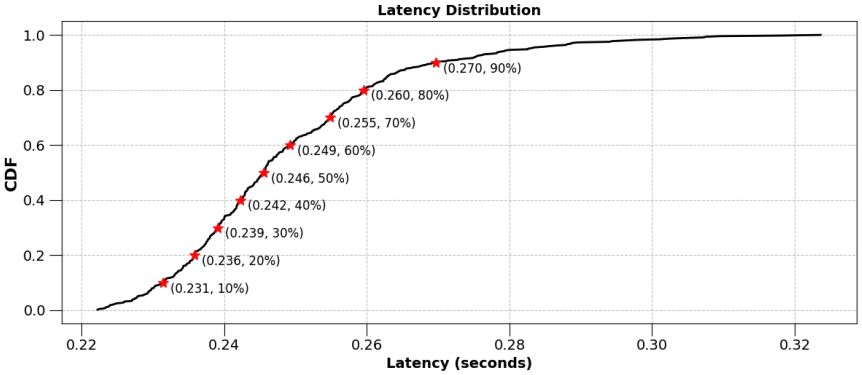

Figure 5: The table summarizes throughput and latency information, while the CDF illustrates the latency distribution of requests with every 10th percentile highlighted.

4 Scoring Your Proxy's Performance

4.1 Grading

Your grade in the MP is based on a set of test cases which will be run by the autograder. The scoring for each the test is based on throughput and latency, with a final score calculated using a weighted combination of these metrics. The following steps outline the computation of the score:

- **Throughput:** The throughput is calculated as the number of successful requests divided by the total test time:

$$\text{Throughput} = \frac{\text{Total Successful Requests}}{\text{Total Time}}$$

- **Latency Metrics:**
  - **Mean latency:** The average latency across all successful requests.
  - **Percentiles:** The 90th, 95th, and 99th percentiles of latency are computed to capture the distribution of latency across requests.
- **Test case performance score:** This score is calculated using a weighted combination

of throughput and latency:

$$\textbf{Test Case Performance Score} = \frac{\text{Throughput}}{\text{Mean Latency} \times 0.9 + \text{95th Percentile Latency} \times 0.1}$$

The higher the throughput and lower the latency (especially the mean latency), the higher the score.

- **Test case grade:** Finally, the score is converted to a grade on this test case by comparing it to our own Staff Proxy implementation which is also run by the autograder on the same test case. The autograder runs 3 trials of each of the two proxy implementations and takes the average credit across iterations. The score difference is mapped to a credit using the following tiered scheme:

Score Difference	Credit
$\leq 15\%$	100%
$\leq 30\%$	80%
$\leq 50\%$	60%
$\leq 70\%$	50%
$> 70\%$	0%

If the throughput is below 50% of the staff proxy's throughput, the credit is 0% regardless of score difference.

- **Your MP grade** is the weighted average of your proxy's test case grades:

$$\text{MP Grade} = \frac{25 \cdot c_1 + 25 \cdot c_2 + 25 \cdot c_3 + 10 \cdot c_4 + 15 \cdot c_5}{100}$$

where c_i is the credit earned on test case i. (The weighting is because the last two test cases are more difficult.)

- **In general, there is no interview for this MP,** as there is insufficient time before the end of the course. Instead, the final exam will have questions about this MP. Also, we reserve the right to require any student to do a code review with the staff, and to adjust grades if there is a lack of demonstrated understanding of the code or evidence of a policy violation (e.g., disallowed use of AI).

4.2 ☆ Leaderboard ☆

For this machine problem, we've introduced a leaderboard to showcase the top student and staff performance scores. Participation in the leaderboard is entirely optional and is not required to achieve a full score on this MP. The leaderboard can be accessed here: `https://netarch.github.io/cs-435-contestboard`.

Students who participate in the leaderboard will be awarded **extra credit** on their final course percent score, as follows: 1st place: 4%; 2nd place: 3%. 3rd place: 2%. 4th-10th place: 1%.

For your submission, in addition to the python file, we expect a credentials.json file in the submission with the following key-value pairs:

- `netid`: Your university NetID.
- `screen_name`: The display name that will appear on the leaderboard alongside your performance score. If the chosen `screen_name` is unavailable, please re-submit with another one.

The format of the json file is as follows:

```
{
    "netid": "Your netid",
    "screen_name": "Your Screen Name"
}
```

The `netid` is **mandatory** for this MP. You may choose whether to provide the `screen_name`: if provided, your submission will be included in the leaderboard. You may pick your own screen name, whether that is your own name or something that reveals nothing about who you are. *Your screen name and score will be publicly visible on the Internet.* We trust you to be responsible in your choice of screen name (i.e., nothing offensive).

The **leaderboard score** is the sum of each test case performance score weighted by the credit earned on the test case:

$$\textbf{Leaderboard Score} = \sum_{i=1}^{5} c_i \cdot \text{score}_i$$

where $c_i \in [0, 1]$ is the credit earned on test case i and score_i is the test case performance score achieved by your proxy on that test case. This is slightly different than the grading so the leaderboard can show finer performance distinctions between implementations. In addition to the score which is used for ranking, the leaderboard will display the throughput, mean latency, and 95% percentile latency (average across test cases).

Please note the following regarding the leaderboard:

- All variables must be provided in `string` format. Your screen name should be limited to letters, numbers, hyphens, underscores, and spaces up to 30 characters.
- If you choose not to appear on the leaderboard, simply omit the `screen_name` key-value pair or set it to an empty string ("screen_name": "").
- If you submit multiple times using different screen names, the leaderboard will simply display the latest screen name alongside your best score across all submissions.
- If your submission appears on the leaderboard and you wish to remove it, simply re-submit without the `screen_name` variable, as mentioned above. This will remove your entry from the leaderboard. Your scores are still stored, so to reappear on the leaderboard, just submit again with an available `screen_name`.

5 Submission Instructions

- Please submit the following files:
  - A python file named `proxy.py` containing your proxy implementation.
  - A json file named `credentials.json` containing your credentials.
  - (Optional) A text file named `requirements.txt` containing a list of dependencies and their version numbers. You can run `pip freeze` within your local `virtual environment` if you've installed additional packages for your full-fledged proxy implementation to update the base requirements.txt file provided within the student package.
  - If you worked in a team, include a file `team.txt` containing the NetIDs of your team, one per line.
- Compress all the files into a single archive named `submission.zip` and upload it.
- **Do not place the files inside a folder before zipping.** Instead, select all files directly and create the zip archive without any enclosing directory.
- The autograding process may take approximately 25 to 30 minutes.

6 Team Formation

Although it is not required, we encourage you to complete this MP in a two-student team. If you choose to work in a group, each student is expected to fully understand the entire codebase and be able to explain and modify any part of it if asked during an interview or code review. Grading will be identical for individual and team submissions. Whether you work in a team or individually, each student must submit the assignment separately from their own account.

7 Generative AI Usage Policy

Use of generative AI tools is permitted to assist students in understanding algorithms and optimizing code (such as debugging). However, solutions may not be generated entirely by AI. Students are expected to write their code independently after gaining a thorough understanding of the underlying algorithms. If you choose to use generative AI tools, you must still be able to justify your design choices and explain all submitted code, including any code generated or suggested by AI or by your teammate.

Good luck and happy coding!